

Chop Chop Serve

Milestone 2 Documentation

Johnno:

Player Controller

This mainly includes the player controls for the chef and the car, additionally the interaction system.

Key Components:

- **InteractableComponent:** Can be attached to any .tscn that's interactable and has signals that can be overridden so the class can have their own implementation on how to be interacted with.
- **Controller:** Although barely implemented, once we start networking this will exclusively handle sending inputs to the server that can be replicated to all the clients

Implementation Status:

Mostly finished- Need to fix the interaction depending on different scenarios and start working getting the player and car fully replicated. Additionally need to start properly implement the control to sync with the server and have interaction synced

UI

Worked on designing the HUD for the level select and cooking gameplay. Currently these are just visuals and are not linked up with the reputation, score or time.

Key Components:

- **UIContents:** Displays the contents inside a plate, pot, etc.. Uses a texture variable stored inside food to know what icon to display
- **UIOrder:** Displays the order of the customer and uses UIProgress to display how long left to accept the order. No functional code inside this yet

- **UIProgress:** Custom progress bar the will be used for majority of the UI that will need to display the time to the player

Implementation Status:

Needs more work on the visual and functional side. I have made only a few UI that are mostly purely visual. Once the controllers are replicated I will start focusing more on the UI aspect.

Nagare:

Appliance System

The system uses an inheritance hierarchy starting with:

- **Placeable** (for positioning/collision/model)
 - **Appliance** (base functionality)
 - **PoweredAppliance** (stoves, ovens, fryers - cooking appliances)
 - **UnPoweredAppliance** (benches, sinks, trash cans - utilities)
 - **Equipment** (pots, pans - portable cooking tools)

Key Components:

- **ApplianceFactory:** Automatically registers and instantiates concrete appliances from the file system
- **ApplianceManager:** Handles appliance lifecycle, team assignment, and resource management
- **Inflammable component:** Generic fire system that can be added to any object for sabotage mechanics
- **Upgrade component:** Generic upgrade system that can enhance any object's properties

Implementation Status:

The core appliance architecture and component systems are complete, still require integration and testing. Prepared extensive console output and documentation for integration. Visual/sound effects need to be implemented in the next polishing stage.

Network Layer

Implemented an abstract base class architecture designed for future expansion (WebSocket for browser hosting), but will likely only implement ENet for this project.

Key Components:

- **NetworkLayer:** Abstract interface defining all networking operations (connection, data transmission, player management)
- **ENetNetworkLayer:** ENet implementation supporting host/client architecture with up to 4 players
- **Lobby:** UI system handling player joining, leaving, and lobby state management with real-time updates

Implementation Status:

The base networking layer and lobby systems are completed. Further integration with game systems needs to wait until we have a solid solo play prototype.

Bradley:

Not a CGRA student but just putting in some context from game design specifically level design and vfx.

Created the level layouts for the game so far, all levels are yet to be tested.

The purpose of making these layouts is so CGRA students can test the gameplay code in a proper gameplay environment.

As for the process into making these layouts, first I sketch up how they are created and then go into godot and place the assets in the same arrangement.

When making a layout the most important part for me is to figure out the flow and how players will use the layout to make orders.

If everything goes well these layouts should be present in the final build as well with potential tweaks.

Also worked on particles so CGRA students can test these early and see if they spawn correctly with their systems.

Mitchell:

Not a CGRA student but just putting in some context from game design specifically 3D modelling and asset creation.

I created the 3D model assets for the food items and the kitchen infrastructure that the player will interact with in the game.

I modelled the assets in Blender, and used a texture map created in Photoshop, and exported the assets in the GLB format, for use in Godot.

I initially created some draft models for Bradley to use to draft out some level layouts, and for the rest of the team to use to test their code, rather than using the default cubes.

I communicated with the team over what systems would be in place, and based what kind of models I would make based on that feedback.

Josh:

My contributions towards the project so far mainly consist of three areas:

Customers using NPC Script (Main Focus): NPC is an “abstract class” I made to build on top of godot’s navigation agent. It defines an NPC with basic movement, pathfinding and object avoidance. Additionally NPC has functions intended to be overridden for more complex behaviour. Customers are a spawnable instance that uses a script extending NPC.

Customers currently do the following:

- Spawn at a chosen location
- Line up in queue (queue size determined by restaurant),
- Located tables if any are free and go to one if customer is first in queue
- Wait at table until they have been served chosen food then move to exit point
- Additionally they will leave to exit point if they are left waiting for too long

There is also a FoodCourt script that helps customers by generating them into the main scene and keeping track of the tables/queue positions and their states (occupied/unoccupied).

Server: The server is an autoloaded script that is meant to act as a central “kernel” for the project. It allows objects to “register as services” which are tracked by the server via a dictionary containing a string id and a reference to the object.

Services can contact the server to call functions from other services. Overall this gives an easy way of managing communications without worrying about parent hierarchical structuring or separation of scenes brought about from dividing work among members. More services are planned to be registered to the server with development beyond milestone two.

Building: I have developed a grid placement system that can be used to move objects on a grid, avoid grid points where the moving object would collide with another and place it down, locking it in place. In future this will be integrated with appliances allowing for custom kitchen layouts decided by players.

Emma:

Have currently worked on the food/cooking system and the order system

- I created a superclass for all the ingredients to use and then each ingredient has its own script which defines its values
- I created all the ingredients but haven't yet paired all the models to the scripts
- I have made it so that the food can change states depending on what appliance the food is being cooked/prepped on
- I have made food that has not been modified be able to spoil
- I have made all the menu items this is basically just saying what set of ingredients and also what previous states of these ingredients allows us to make the menu item e.g. tomato soup
- I have made the plate so that when ingredients are picked up on the plate it will do a check and find a matching menu item with the correct ingredients and make sure they have gone through all the proper preparing states to make the menu item. E.g. tomatoes must be chopped and then boiled in water to make tomato soup.
- I have started on the generating of orders, it is supposed to select from a list of available food that can be made or has learnt to be made. This allows the players to not be bombarded with too many different things at once.
- The generate order function returns 2 starters, 2 mains and a desert

Jessica:

So far I have worked on creating the Currency System, Reputation System, Game Phases and the Sabotage System.

Currency and Reputation Systems:

Scripts to help with the currency and reputation of the player throughout the game.

Key Components:

- **Check function:** Used to see if the player has enough currency to buy something or make sure their reputation isn't exceeding the limit.
- **Add (and Minus):** Removes or adds currency/ reputation for the player. It calls the check function each time.

Implementation Status:

They work and include everything they should need but when we add a second player/ team to the game these scripts will need to be updated to handle that. You can see them working by pressing the buttons on the screen, but these will be removed later. They also need to be used to update the HUD UI on the screen in the future.

Game Phases:

The Different Phases of the Game - Prep, cook, Sabotage etc. Basic design created but not within the main scene and has lots of work to be done on it still.

Sabotage System:

A Set of different 'challenges' you can send to the other team. Have implemented a basic Water Spill sabotage currently.

Key Components:

- **Calling Sabotage:** Press a button to call the sabotage with an enum of the different sabotages.
- **WaterSpill object:** The water object that turns up within the game. It is randomly placed within the scene.
- **The Consequences:** When the Player walks through the waterSpill they lose some reputation.

Implementation Status:

Overall there is still a lot to do with the SabotageSystem, so far we just have a simple waterSpill sabotage. For which there is a button the player can press to create the spill if they have enough money. Then when the player walks through it they lose reputation. I still need to create the limitations on player movement within the spill as well as the random chance a customer will fall when in one. *This hasn't been implemented within the main scene yet, but can be seen from the JessTestScene.*

Video Demo Link: <https://youtu.be/P1bXwwFyT1g>

Restaurant Game Prototype - Quick Start Guide

Controls

- **WASD** - Move player in 8 directions
- **SPACE or E** - Interact (pick up/place items)
- **J** - Action button (for chopping)

Game Area

The prototype shows a restaurant with 4 kitchens. **Only the bottom-right kitchen is functional** - the others are placeholder models.

Basic Gameplay

Picking Up Items

Stand in front of an item and press **SPACE**:

- Get tomatoes from the food crate
- Get water from the sink

Using Appliances

Stand in front of an appliance with a valid item and press **SPACE** to place:

- **Pot**: Can hold tomato + water
- **Chopping Board**: Can hold tomato only

Cooking Process

- Most appliances cook automatically once food is placed
- **Exception - Chopping Board**: Stand in front with food on it, then **hold J** to chop

Making Dishes

1. Get a plate
2. Stand in front of an active appliance (like a pot with cooked food)
3. Press **SPACE** to transfer food from appliance to plate

Quick Recipe Example

1. Pick up tomato from crate
2. Pick up water from sink
3. Place both in pot (cooks automatically)
4. Get plate
5. Take cooked food from pot to plate

Extra Demo Controls:

To showcase the features for key appliance we use the follow Numpad controls in the video:

- Numpad 1 for spawning a stove with a pot
- Numpad 5 for spawning a chopping table
- Numpad 7 for spawning a food crate